

Lab 9 Pair Programming – Polymorphism and Exception Handling

Aim

This week is focused on the Polymorphism and Exception Handling. You will start using Pair Programming to learn more from each other.

One effective method for cultivating computational thinking skills while learning programming in computer labs is through Pair Programming. In this approach, two students collaborate on solving the same problem. One student assumes the role of the Driver, responsible for writing the code and translating design concepts into actual code implementation. Meanwhile, the other student, known as the Observer, reviews the code, identifies errors, and provides design suggestions and improvements. The roles of the driver and observer are continuously switched throughout the session.

在计算机实验室学习编程时，培养计算思维技能的一种有效方法是采用“结对编程”。在这种方法中，两名学生合作解决同一个问题。一名学生扮演“驾驶员”的角色，负责编写代码并将设计概念转化为实际的代码实现。与此同时，另一名学生，被称为“观察员”，他审查代码，识别错误，并提供设计、改进建议。在整个会话中，“驾驶员”和“观察员”的角色不断交替。

Pair Programming Lab:

1. The Lab TA will create a pair within the group.
2. Talk to your pair, who is going to be the Driver first.
3. Complete Task 1 with Pair Programming (30-45 minutes).
4. Show the TA the results of your works in Task 1.
5. Switch roles.
The Observer becomes the Driver and vice versa.
6. Complete Task 2 with Pair Programming (30-45 minutes).
7. Report to TA after you have completed Task 2.
8. TA gives grade.

Pair Programming Lab 9 Task 1: Draw different shapes in a picture

1. Open your preferred Java Integrated Development Environment (IDE) or a text editor to write your Java code.
2. Create a new Java file named Shape.java. This file will contain the superclass **Shape**. Here, **Shape** is a simple class with a method **draw()**, which will serve as the method to be overridden in subclasses.
3. Create a new Java file for each subclass. For instance, **Circle.java**, **Square.java**, and **Triangle.java**.
4. In each file, define the subclass that extends the **Shape** superclass and overrides the **draw()** method.
5. Create a new Java file, let's call it **Main.java**. In this file, write the **main()** method to demonstrate polymorphism.
6. An array named **picture** is declared and initialized to hold **Shape** objects.
7. Objects of **Circle**, **Square**, and **Triangle** are created and assigned to respective indexes in the

picture array.

8. A loop iterates through the picture array and calls the **draw()** method for each object in the array, resulting in the respective shapes being drawn based on their overridden **draw()** methods.

This way, the picture array holds various shapes, and by using a loop, you can perform operations on each shape without knowing the specific subclass at compile time, thus demonstrating the power of polymorphism.

Pair Programming Lab 9 Task 2: Throwing and catching exceptions

1. Open your preferred Java Integrated Development Environment (IDE) or a text editor to write down the following Java code (with appropriate formatting)

```
public static char findCharAt(String str, int idx) {  
    return str.charAt(idx);  
}  
  
public static void main(String[] args) {  
    String str = "Hello World";  
    char charAtNegativeIndex = findCharAt(str, -1); // Trying to access at negative index  
    char charAtLengthIndex = findCharAt(str, 11); // Trying to access at index equal to size  
}
```

2. Check with the **class of Exception** returned by the IDE
3. Redesign the **findCharAt** method to simulate the throwing of the same exception with corresponding **if blocks** to check the exception conditions
4. Redesign the **main** method with both try and catch blocks to handle this exception
5. The handling the exception should print two lines, the first line is the **information of exception** type, and the second line is **the reason of this exception**
6. Carefully design the **try and catch blocks** to catch **both exceptions** in the main method (hint: use more than one try-catch block structure)